

Computer Vision Homework 5

NTU CSIE D13922014 黃丰楷

(a) Dilation

- Description

I developed a grayscale dilation function that processes each pixel in the input image `img`. For each pixel with an intensity greater than zero, the function examines its neighboring pixels, as defined by a kernel, and identifies the highest intensity value among them. If a neighboring pixel falls within the image boundaries, its coordinates are recorded, and those positions in `img_temp` are updated with this maximum intensity. The outcome is a dilated grayscale image, where bright regions expand according to the highest local intensity values.

- Code

```
def dilation(img, kernel):
    img_temp = np.zeros(img.shape)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            if img[i][j] > 0:
                maximum = 0
                idx = []
                jdx = []
                for (k, l) in kernel:
                    i_d = i + k
                    j_d = j + l
                    if i_d >= 0 and j_d >= 0 \
                        and i_d <= (img.shape[0]-1) and j_d <= \
                            (img.shape[1]-1):
                        maximum = max(maximum, img[i_d][j_d])
                        idx.append(i_d)
                        jdx.append(j_d)
                img_temp[(idx, jdx)] = maximum

    return img_temp
```

- Result



(b) Erosion

- Description

I created a grayscale erosion function that processes each pixel in the input image `img`. For each pixel, the function inspects neighboring pixels according to a specified kernel. If any neighboring pixel is out of bounds or has a zero intensity, the central pixel is left unchanged. Otherwise, the function identifies the minimum intensity among the neighbors, records the coordinates, and updates the corresponding locations in `img_temp` to this minimum intensity. The result is an eroded grayscale image where darker areas shrink, guided by the lowest intensity values in the neighborhood.

- Code

```
def erosion(img, kernel):
    img_temp = np.zeros(img.shape)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            draw = True
            minimum = 255
            idx = []
            jdx = []
            for (k, l) in kernel:
                i_d = i + k
                j_d = j + l
                if i_d < 0 or j_d < 0 \
                    or i_d > (img.shape[0]-1) or j_d > (img.shape[1]-1) \
                    or img[i_d][j_d] <= 0:
                    draw = False
                    break
            else:
                idx.append(i)
                jdx.append(j)
                minimum = min(minimum, img[i_d][j_d])
            if draw:
                img_temp[(idx, jdx)] = minimum
```

```
return img_temp
```

- Result



(c) Opening

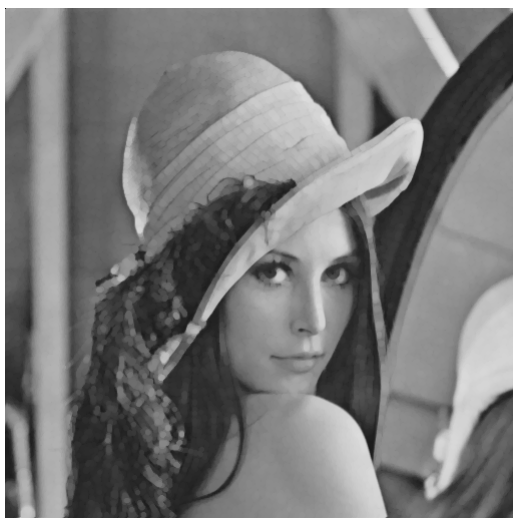
- Description

I implemented a grayscale opening function, which can be formulated as $B \circ K = (B \ominus J) \oplus K$.

- Code

```
def opening(img, kernel):  
    return dilation(erosion(img, kernel), kernel)
```

- Result



(d) Closing

- Description

I implemented a grayscale closing function, which can be formulated as $B \bullet K = (B \oplus J) \ominus K$.

```
def closing(img, kernel):  
    return erosion(dilation(img, kernel), kernel)
```

- Result

